

Embedded System Software 과제 1

(과제 수행 결과 보고서)

과목명: [CSE4116] 임베디드시스템소프트웨어

담당교수: 서강대학교 컴퓨터공학과 박 성 용

학번 및 이름: 20181688, 조태연

개발기간: 2024. 05. 03. -2024. 05. 19.

최 종 보 고 서

I. 개발 목표

- 디바이스 드라이버와 타이머 기능을 포함한 하나의 module을 구현한다.

II. 개발 범위 및 내용

가. 개발 범위

- 디바이스 드라이버와 타이머 기능을 포함한 하나의 module
- module을 사용하는 응용 프로그램

나. 개발 내용

- timer_driver.c

Timer_driver.c는 디바이스를 등록 및 등록 해제하고 open(),release(),init(),exit()와 같은 기본에 더불어 유저 프로그램으로부터 입력받은 ioctl()과 read()를 처리하는 기능을 구현했다.

- device.c

Device.c는 보드의 FPGA 디바이스들을 다뤄야 한다. 구현해야 할 디바이스는 Dot, FND, LED, LCD이고 RESET은 timer_driver.c에서 직접 구현했다.

- app.c

사용자로부터 입력받은 인자들을 ioctl로 드라이버에 전달하고 read()를 통해 RESET 스위치값을 읽어와서 스위치가 눌리면 timer가 동작하게끔 ioctl()함수를 사용하게 구현했다.

III. 추진 일정 및 개발 방법

가. 추진 일정

- 05.08~05.09 관련내용 학습
- 05.09~05.12 디바이스에 값을 출력하는 device.c 작성
- 05.13~05.16 타이머기능과 device.c에 있는 함수를 호출하는 timer_driver.c 작성

-05.17~05.18 최종 디버깅 및 테스트

-05.19 보고서 작성

나. 개발 방법

- timer_driver.c

```
static struct file_operations dev_driver_fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .read = iom_fpga_dip_switch_read,
    .release = device_release,
    .unlocked_ioctl = device_ioctl,
};
```

디바이스 드라이버는 dev_driver_fops 구조체에 지정된 함수들을 토대로 동작한다.

```
static int device_open(struct inode *inode, struct file *file){

    if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
        return -EBUSY;

    printk(KERN_INFO "Device open\n");

    try_module_get(THIS_MODULE);
    return SUCCESS;
}

static int device_release(struct inode *inode, struct file *file){

    printk(KERN_INFO "Device release\n");

    atomic_set(&already_open, CDEV_NOT_USED);

    module_put(THIS_MODULE);
    return SUCCESS;
}
```

Open()은 디바이스를 여는 역할을 하는데 이때 열려 있다면 이미 사용중이라면 -EBUSY를 반환한다. 사용중이 아니라면 try_module_get()을 통해 카운트를 증가시킨다.

Release()는 open의 반대 역할을 수행한다.

```

static long device_ioctl(struct file *file, unsigned int ioctl_num, unsigned long ioctl_param) { //app에서 ioctl이용시 사용되는 함수

    int temp,current_pos,num;

    if(ioctl_num==COMMAND){ //timer시작

        my_timer.expires=get_jiffies_64();
        my_timer.data = (unsigned long)&count;
        my_timer.function=timer_handler;
        add_timer(&my_timer);
    }
    else if (ioctl_num==SET_OPTION){ //입력값 받아와서 화면에 입력 값 출력하고 timer 초기화

        copy_from_user(&val, (value*)ioctl_param, sizeof(value));
        init_timer(&my_timer);
        mode=1;
        pos_lcd=15;
        count=0;

        temp=val.timer_init;
        while(temp>10){
            temp/=10;
        }

        num = (temp)%8;
        if(num==0) num=8;

        fnd_device(val.pos,num);
        led_device(num);
        dot_device(num);
        lcd_device(-1);
    }
    else{

        pr_alert("Enter the correct ioctl_num\n");
    }

    return SUCCESS;
}

```

응용프로그램에서 드라이버를 사용하는 ioctl()을 위한 함수이다. 명세서에서 COMMAND와 SET_OPTION두 명령을 구분해서 사용하게끔 했는데 SET_OPTION 명령을 하는 경우에는 응용프로그램 동작 시 입력되는 값들과 관련된 구조체인 val의 입력을 읽어오고 my_timer를 초기화 한 후에 초기 입력값을 화면에 출력한다. COMMAND 명령을 하는 경우에는 my_timer를 시작하도록 한다.

```

ssize_t iom_fpga_dip_switch_read(struct file *inode, char *gdata, size_t length, loff_t *off_what) { //별도로 reset버튼 입력만을 확인하기 위한 함수

    unsigned char dip_sw_value;
    unsigned short _s_dip_sw_value;

    if (!iom_fpga_dip_switch_addr ) {
        pr_err("iom_fpga_addr[RESET] is not mapped\n");
        return 0;
    }

    _s_dip_sw_value = inw((unsigned int)iom_fpga_dip_switch_addr);

    dip_sw_value = _s_dip_sw_value & 0xFF;

    if (copy_to_user(gdata, &dip_sw_value, 1))
        return -EFAULT;

    return length;
}

```

응용프로그램에서 사용하는 read()를 위한 함수로 reset 스위치의 상태를 읽어오게 해준다.

```

void timer_handler(unsigned long data){ //timer_handler

    int *cnt=(int*)data;
    int current_pos,num;
    int temp;
    temp=val.timer_init;
    //timer_cnt가 0이 됐을때부터의 종료 시퀀스 expires를 1초로 설정하고 출력해야 할 문구에서 숫자부분만 3 2 1로 바꾼 후 화면 모두 빈칸
    if(*cnt==val.timer_cnt){

        fnd_device(0,0);
        led_device(0);
        dot_device(-1);
        lcd_device(-4);
        (*cnt)++;
        my_timer.expires=get_jiffies_64()+10*HZ/10;
        my_timer.data=(unsigned long)cnt;
        my_timer.function=timer_handler;
        add_timer(&my_timer);
    }
    else if(*cnt==val.timer_cnt+1){

        lcd_device(-3);
        (*cnt)++;
        my_timer.expires=get_jiffies_64()+10*HZ/10;
        my_timer.data=(unsigned long)cnt;
        my_timer.function=timer_handler;
        add_timer(&my_timer);
    }
    else if(*cnt==val.timer_cnt+2){

        lcd_device(-2);
        (*cnt)++;
        my_timer.expires=get_jiffies_64()+10*HZ/10;
        my_timer.data=(unsigned long)cnt;
        my_timer.function=timer_handler;
        add_timer(&my_timer);
    }
    else if(*cnt==val.timer_cnt+3){

        lcd_device(-5);
        return 0;
    }
}

```

```

}
else{ //동작중일때
    //처음에 init에서 0이 아닌 숫자가 무엇이 입력되었는지 체크
    while(temp>10){
        | temp/=10;
    }

    //몇번 timer가 지나갔는지를 이용하여 현재 위치와 표시해야할 숫자 계산
    current_pos=(val.pos+(*cnt/8))%4;
    num = (temp+*cnt)%8;
    if(num==0) num=8;
    //device에 출력
    fnd_device(current_pos,num);
    led_device(num);
    dot_device(num);
    lcd_device(*cnt);
    //timer 세팅
    (*cnt)++;
    my_timer.expires=get_jiffies_64()+val.timer_interval*HZ/10;
    my_timer.data=(unsigned long)cnt;
    my_timer.function=timer_handler;
    add_timer(&my_timer);
}
}
}

```

My_timer의 timer_handler()이다. If ~ else if()까지는 정해진 timer_cnt만큼 흐른 후에 종료 시퀀스이다. Else는 동작 중일 때 fpga 디바이스들을 이용하여 화면에 출력하고 timer세팅을 하는 부분이다.

```
static int __init dev_driver_init(void){
    int register_result = register_chrdev(DEVICE_MAJOR_NUMBER, DEVICE_NAME, &dev_driver_fops);

    if (register_result < 0) {
        pr_alert("Registering dev_device failed with %d\n", register_result);
        return register_result;
    }
    //디바이스의 물리주소를 매핑//
    iom_fpga_init();
    iom_fpga_dip_switch_addr = ioremap(IOM_FPGA_DIP_SWITCH_ADDRESS, 0x1);

    pr_info("Assigned major number : %d.\n", DEVICE_MAJOR_NUMBER);
    cls = class_create(THIS_MODULE, DEVICE_NAME);
    device_create(cls, NULL, MKDEV(DEVICE_MAJOR_NUMBER, 0), NULL, DEVICE_NAME);
    pr_info("Device created on /dev/%s\n", DEVICE_NAME);
    return SUCCESS;
}

static void __exit dev_driver_exit(void){

    //해제 후 timer삭제
    iom_fpga_exit();
    del_timer(&my_timer);

    device_destroy(cls, MKDEV(DEVICE_MAJOR_NUMBER, 0));
    class_destroy(cls);

    unregister_chrdev(DEVICE_MAJOR_NUMBER, DEVICE_NAME);
    pr_info("Exit device on /dev/%s\n", DEVICE_NAME);
}
```

Insmod()와 rmmod()로 모듈이 커널에 설치되었다가 삭제 될 때 각각 사용되는 init()함수와 exit()함수이다.

-device.c

```
void iom_fpga_init(void){ //디바이스의 물리 주소를 mapping

    iom_fpga_addr[FND]=ioremap(IOM_FND_ADDRESS, 0x4);
    iom_fpga_addr[LED]=ioremap(IOM_LED_ADDRESS, 0x1);
    iom_fpga_addr[DOT]=ioremap(IOM_FPGA_DOT_ADDRESS, 0x10);
    iom_fpga_addr[LCD]=ioremap(IOM_FPGA_LCD_ADDRESS, 0x32);
}

void iom_fpga_exit(void){ //해제

    int i;
    for(i=0;i<DEVICE_NUM;i++) iounmap(iom_fpga_addr[i]);
}
```

각 디바이스들의 물리주소를 맵핑하고 해제하는 함수이다.

```
int fnd_device(int pos,int num){ //pos와 num를 이용해서 fnd에 값 출력

    unsigned char value[4]={0,};
    unsigned short int value_short=0;

    value[pos]=num;
    value_short=value[0] << 12 | value[1] << 8 | value[2] << 4 | value[3];
    outw(value_short,(unsigned int)iom_fpga_addr[FND]);

    return 0;
}
```

4자리 중에 어느 위치에 있어야 하는지, 어떤 숫자를 띄워야 하는지를 pos와 num으로 입력받아 FND 디바이스에 숫자를 출력해준다.

```
void led_device(int num){ //num을 led에 출력

    unsigned short _s_value=0;

    _s_value=1<<(8-num);
    if(num==0) _s_value=0;
    outw(_s_value,(unsigned int)iom_fpga_addr[LED]);
}
```

입력된 num에 따라 LED 디바이스에 입력된 숫자 번호에 불이 들어오게 한다.

```

void dot_device(int num){ //num을 dot에 출력 만약 -1입력시 빈칸 출력

    int i=0;
    unsigned short _s_value;
    if(num==-1){
        for(i=0;i<10;i++){
            _s_value=fpga_set_blank[i] & 0x7F;
            outw(_s_value,(unsigned int)iom_fpga_addr[DOT]+i*2);
        }

        return;
    }
    for(i=0;i<10;i++){
        _s_value=fpga_number[num][i] & 0x7F;
        outw(_s_value,(unsigned int)iom_fpga_addr[DOT]+i*2);
    }
    return;
}

```

입력된 num에 따라 DOT MATRIX에 숫자를 출력한다.


```

void lcd_device(int cnt){ //timer가 진행된 횟수 cnt를 입력받아 lcd에 출력

    unsigned short int _s_value = 0;
    unsigned char output[MAX_BUFF+1] = {'\0'};
    int i, timer_cnt_len = 0;
    int temp = val.timer_cnt;

    while (temp > 0) { //출력해야할 timer_cnt가 몇자리수인지 계산
        temp /= 10;
        timer_cnt_len++;
    }
    if (cnt == -5) { //-5입력시 빈칸으로 초기화
        memset(output, ' ', MAX_BUFF);
    } else if (cnt <= -2 && cnt >= -4) { //-4 ~ -2 입력시 종료 후 문구 출력

        memset(output, ' ', MAX_BUFF);
        strncpy(output, "Time's up!      0", LINE_BUFF);
        cnt=(-1)*cnt-1;
        sprintf(output + 16, MAX_BUFF - 15, "Shutdown in %d...",cnt);
    } else { //timer 동작시 문구 출력
        //첫번째줄 출력 확인 입력 후 오른쪽 정렬하여 남은 timer_cnt출력
        memset(output, ' ', MAX_BUFF);
        strncpy(output, "20181688", MAX_BUFF);
        if(cnt==1) sprintf(output + 8, MAX_BUFF - 8, "%*d", 16 - 8, val.timer_cnt );
        else sprintf(output + 8, MAX_BUFF - 8, "%*d", 16 - 8, val.timer_cnt - cnt);

        //두번째줄 출력 pos_lcd와 mode를 이용하여 이니셜의 처음위치와 현재 상황이 오른쪽으로 진행해야 하는지 왼쪽으로 진행해야 하는지를 체크
        if (mode == 1) { // 오른쪽으로 이동
            if (pos_lcd < 29) {
                pos_lcd++;
            } else {
                mode = 0; // 방향 전환
                pos_lcd--;
            }
        } else { // 왼쪽으로 이동
            if (pos_lcd > 16) {
                pos_lcd--;
            } else {
                mode = 1; // 방향 전환
                pos_lcd++;
            }
        }

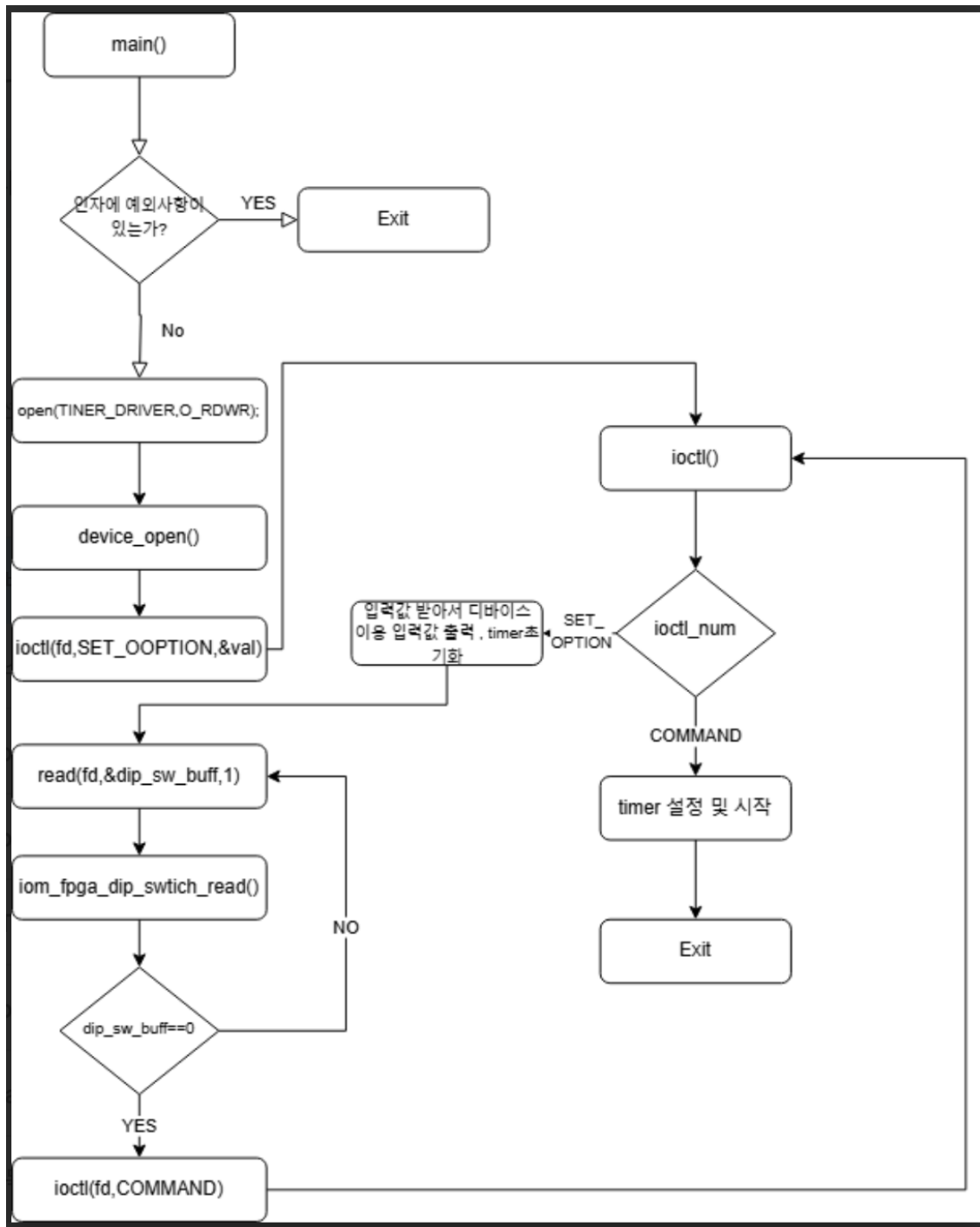
        strncpy(output+16,"                ",16);
        strncpy(output + pos_lcd, "JTY", 3);
    }

    for (i = 0; i < MAX_BUFF; i += 2) {
        _s_value = (output[i] & 0xFF) << 8 | (output[i + 1] & 0xFF);
        outw(_s_value, (unsigned int)iom_fpga_addr[LCD] + i);
    }
}

```

My_time가 진행된 횟수인 cnt를 입력받아 상황에 맞는 text를 LCD 디바이스에 출력하는 함수이다.

-app.c



IV. 연구 결과

App 실행시에 입력하는 3개의 값에 대한 예외처리가 되었고 실행시키면 입력한 값을 디바이스에 출력한 상태로 대기하다가 reset버튼을 누르면 명세서에 써 있는 대로 동작 후에 종료 시퀀스를 거쳐서 종료 후 모두 빈칸 상태로 초기화한다.

V. 기타

과제 1에 비해 볼륨이 작아 큰 문제는 없었는데 교수님이 수업시간에 말씀해주신 것처럼 driver쪽 함수에서 오버플로우 등 코드 상의 오류가 발생하면 kernel panic이 발생해버려 보드가 꺼지는 경우가 발생해서 디버깅하는데 번거로움이 있었다. 교수님의 강의와 실습으로 배운 내용을 계속 보게 되면서 이해가 더 깊어졌다.